# Sanity Checks — Test Your Game Logic

**Day 05**
Lesson 05

**Nora XIE**
Course 02

**Phase 1 — Game Studio** **Time:** 60–75 min **Mode:** Self-study

**Today's goals:**

- Write small check functions that return True/False
- Test edge cases: 0 HP, no poison left
- Run checks before playtesting

---

Course 02 quest map: Day 05 of 18

## Hook

Before you balance a game, prove the math cannot go negative or break.

## Learn

### Learn

A **check** is a tiny function with no `input()` — just logic.

```
def check_poison_uses(uses):
    return uses >= 0

assert check_poison_uses(1)  # crashes if False
```

Good checks: HP never below 0; poison uses decrease by 1 only; defend reduces damage.

### AI rules today

**Predict/Run:** AI off. **Investigate:** Socratic hints only. **Make:** you review every line.

Predict → Run → Investigate → Modify → Make

### Demo code

**Folder:** `day05/`

```python
def clamp_hp(hp):
    return max(0, hp)

def check_clamp():
    assert clamp_hp(-5) == 0
    assert clamp_hp(10) == 10
    print("All checks passed.")

check_clamp()
```

### Try it

In terminal, `cd course02/day05`, then predict output, then run: `python demos/demo_checks.py`

### Assignment

**Work in:** `day05/assignment/`

**A** — **Predict** What happens when `assert False` runs?
**B** — **Debug** `assignment/buggy_checks.py` — wrong comparison.
**C** — **Level up** Write 3 checks for your game in `assignment/test_game.py`. Run them with `python test_game.py`.
**D** — **Explain** One bug a check caught (or could catch) in your game.

### Checklist

- ☐ Re-read **Learn**; predict outputs (10 min)
- ☐ Assignments A–D (35–45 min)
- ☐ Run your program 3 times with different inputs
- ☐ Stuck 15 min → C-R-A-V (`../shared/prompts.md`)

## Reflection

1. Easiest part today?
2. One line you can teach a friend.

**Tomorrow:** Mini project: data-driven boss from JSON.